Approximation Algorithms

Week 14 – Algorithms

Korea University Sejong Campus, Dept. of Computer Science & Software

Part 1. Approximation Algorithms – Foundations

Why Approximation Algorithms?

NP-complete problems appear everywhere in the real world, yet no polynomial-time algorithm has been found for any of them.

To deal with NP-complete problems, **we must give up one** of the following three:

1. Finding a solution **in polynomial time**
2. Finding a solution **for all inputs**
3. Finding the **optimal** solution

Approximation algorithms give up (3) – they find a near-optimal solution in polynomial time.

Approximation Ratio

An approximation algorithm must come with an **approximation ratio** that tells us how close the approximate solution is to the optimal solution.

$$\text{Approximation Ratio} = \frac{\text{Approximate Solution Value}}{\text{Optimal Solution Value}}$$

- A ratio closer to **1.0** means higher accuracy
- **Problem**: Computing the ratio requires knowing the optimal solution (which is what we cannot find!)
- **Solution**: Use an **indirect optimal solution** – a value we can compute that is guaranteed to be a bound on the true optimal

Indirect Optimal Solution – Key Idea

For **minimization** problems:

- Find a computable value **L** such that **OPT >= L** (lower bound)
- Show that the approximate solution **APX <= c * L**
- Then **APX <= c * OPT**, giving approximation ratio **c**

For **maximization** problems:

- Find a computable value **U** such that **OPT <= U** (upper bound)
- Show that the approximate solution **APX >= U / c**
- Then **APX >= OPT / c**

1. Traveling Salesman Problem (TSP)

TSP – Problem Definition

Traveling Salesman Problem (TSP)

- A salesperson starts from a city, visits every other city exactly once, and returns to the starting city
- **Goal**: Minimize the total travel distance

Conditions (Metric TSP):

- **Symmetry**: distance(A, B) = distance(B, A)
- **Triangle inequality**: distance(A, B) <= distance(A, C) + distance(C, B)

These conditions hold for Euclidean distances and many practical scenarios.

MST-Based Approach – Intuition

To design a polynomial-time approximation, find a related problem with a known efficient algorithm:

Minimum Spanning Tree (MST) shares key properties with TSP:

- MST connects **all vertices** (like TSP visits all cities)
- MST minimizes **total edge weight** (related to minimizing tour length)

We leverage MST to construct an approximate TSP tour.

Approx_MST_TSP – Algorithm

Input: n cities with pairwise distances

Output: A tour visiting each city exactly once and returning to the start

1. Compute the **MST** of the input graph (using Kruskal's or Prim's)
2. Starting from any city, perform a **DFS traversal** of the MST, recording the visit order (each edge traversed twice)
3. **Remove duplicate cities** from the visit order (keep the first occurrence of each city, plus the starting city at the end)

Return the resulting tour

Worked Example – Step 1: Build MST

Consider 8 cities: **A, B, C, D, E, F, G, H**

```
Original Graph          MST (computed by Kruskal/Prim)

A --- B --- C               A
|   / |   / |              / \
|  /  |  /  |             G   (other edges)
| /   | /   |            / \
D --- E --- F            E   D
|         / |              / \
|        /  |             H   F
G ------H   |            / \   \
             |           C   B   ...
```

Apply Kruskal's or Prim's algorithm to find the MST.

Worked Example – Step 2: DFS Traversal

Starting from city **A**, traverse the MST edges (each edge used exactly twice):

Full traversal order (with backtracking):

$$A \to G \to E \to G \to D \to H \to C \to H \to D \to F \to B \to F \to D \to G \to A$$

Total distance of this traversal = **2M** (where M = total MST weight)

because each of the (n-1) tree edges is traversed exactly **twice** (once forward, once back).

Worked Example – Step 3: Remove Duplicates

Apply triangle inequality to shortcut through already-visited cities:

$$A \to G \to E \to \cancel{G} \to D \to H \to C \to \cancel{H} \to \cancel{D} \to F \to B \to \cancel{F} \to \cancel{D} \to \cancel{G} \to A$$

Resulting tour: A -> G -> E -> D -> H -> C -> F -> B -> A

By the **triangle inequality**, each shortcut is no longer than the path it replaces.

Therefore, the tour length **<= 2M**.

TSP – Approximation Ratio Proof

Claim: Approx_MST_TSP has approximation ratio **<= 2.0**

Proof:

Let **M** = total weight of MST, **OPT** = optimal TSP tour length, **APX** = approximate tour length.

1. **OPT > M**: The optimal tour visits all cities and returns to start. Removing any edge from the optimal tour gives a spanning tree. Since MST is the minimum spanning tree, M <= (tour minus one edge) < OPT.

2. **APX <= 2M**: The DFS traversal has length exactly 2M. Shortcutting via triangle inequality can only decrease the length.

3. **Combining**: APX <= 2M < 2 * OPT

$$\frac{APX}{OPT} < 2.0$$

TSP – Time Complexity

Step	Operation	Complexity
1	Build MST (Kruskal's)	O(m log m)
1	Build MST (Prim's)	O(n^2)
2	DFS traversal of MST	O(n)
3	Remove duplicates	O(n)

Overall: Dominated by MST construction

- Kruskal's: **O(m log m)** where m = number of edges
- Prim's: **O(n^2)** where n = number of cities

2. Vertex Cover Problem

Vertex Cover – Problem Definition

Vertex Cover of a graph G = (V, E):

- A subset S of V such that **every edge** in E has at least one endpoint in S
- **Goal**: Find the **minimum-size** vertex cover

Real-world analogy: Placing the minimum number of CCTV cameras at intersections so that every corridor (edge) is monitored.

Example: For a triangle graph {1, 2, 3}:

- {1, 2, 3}, {1, 2}, {1, 3}, {2, 3} are all vertex covers
- {1} is the minimum vertex cover if vertex 1 connects to all edges
- {2} alone fails if edge (1,3) is uncovered

Maximal Matching Approach

Key insight: Instead of the expensive set cover reduction, use **maximal matching**.

Matching: A set of edges with no shared endpoints

Maximal Matching: A matching where no more edges can be added

- Not the same as *maximum* matching (which is the largest possible)
- A maximal matching can be found greedily in polynomial time

Idea: Select edges greedily (if neither endpoint is already covered), then take all endpoints as the vertex cover.

Approx_Matching_VC – Algorithm

Input: Graph G = (V, E)

Output: A vertex cover

1. Find a **maximal matching** M in G:
   - For each edge (u, v) in E:
     - If neither u nor v is an endpoint of an already-selected edge, add (u, v) to M
2. **Return** the set of all endpoints of edges in M

Worked Example – Vertex Cover

```
Given graph with 9 vertices:
   1 --- 2 --- 3
   |   / |   / |
   4 --- 5 --- 6
   |   / |   / |
   7 --- 8 --- 9
```

Maximal matching M (selected edges shown): edges a, b, c, d, e, f

- Approximate solution: **12 endpoints** (all endpoints of 6 matching edges)
- Optimal solution: **7 vertices**
- Ratio: 12/7 ~ 1.71 (within the guaranteed bound of 2.0)

Vertex Cover – Approximation Ratio Proof

Claim: Approx_Matching_VC has approximation ratio **= 2.0**

Proof:

Let |M| = number of edges in the maximal matching, OPT = size of optimal vertex cover.

1. **OPT >= |M|**: Any vertex cover must include at least one endpoint of each matching edge. Since matching edges share no endpoints, at least |M| distinct vertices are needed.

2. **APX = 2|M|**: The algorithm returns both endpoints of every matching edge.

3. **Combining**:

$$\frac{APX}{OPT} = \frac{2|M|}{OPT} \leq \frac{2|M|}{|M|} = 2.0$$

Vertex Cover – Time Complexity

Finding a maximal matching:

- For each edge, check if its endpoints are already used: **O(n)** per edge
- Total edges: m

Time Complexity: **O(nm)**

3. Bin Packing Problem

Bin Packing – Problem Definition

Bin Packing Problem:

- Given n items with sizes s_1, s_2, …, s_n and bins of capacity **C**
- Each item size <= C
- **Goal**: Pack all items into the **fewest number of bins**

This is an NP-hard optimization problem with many practical applications (memory allocation, container loading, disk partitioning).

Four Greedy Heuristics

Method	Strategy
First Fit (FF)	Scan bins from the first; place item in the **first bin with enough room**
Next Fit (NF)	Check only the **most recently used bin**; if it fits, place it there; otherwise open a new bin
Best Fit (BF)	Place item in the bin where it fits with the **least remaining space**
Worst Fit (WF)	Place item in the bin where it fits with the **most remaining space**

For all four methods: if no existing bin has room, **open a new bin**.

Worked Example – Bin Packing

C = 10, items = [7, 5, 6, 4, 2, 3, 7, 5]

Method	Bin Contents	Bins Used
First Fit	{7,3}, {5,4}, {6,2}, {7}, {5}	5
Next Fit	{7}, {5}, {6,4}, {2,3}, {7}, {5}	6
Best Fit	{7,3}, {5,4}, {6,2}, {7}, {5}	5
Worst Fit	{7,2}, {5,4}, {6,3}, {7}, {5}	5
Optimal	{7,3}, {5,5}, {6,4}, {7,2}	**4**

Approx_BinPacking – Algorithm

Input: n items with sizes s_1, …, s_n

Output: Number of bins used B

```
B = 0
for i = 1 to n:
    if there exists a bin with enough room (by greedy strategy):
        place item i in that bin
    else:
        open a new bin
        place item i in the new bin
        B = B + 1
return B
```

Bin Packing – Time Complexity

Method	Per-item work	Total
First Fit	Scan all bins: O(n)	**O(n^2)**
Best Fit	Scan all bins: O(n)	**O(n^2)**
Worst Fit	Scan all bins: O(n)	**O(n^2)**
Next Fit	Check one bin: O(1)	**O(n)**

Bin Packing – Ratio Proof (FF, BF, WF)

Claim: First Fit, Best Fit, and Worst Fit all have approximation ratio **<= 2.0**

Key observation: At most **one bin** can be less than half full.

- If two bins were each less than half full, their items could fit in one bin, contradicting the greedy strategy.

Proof:

Let OPT = optimal number of bins, OPT′ = number of bins used by the algorithm.

- OPT >= (sum of all item sizes) / C
- At least (OPT′ - 1) bins are more than half full:

$$\sum s_i > (OPT' - 1) \times \frac{C}{2}$$

$$\frac{\sum s_i}{C} > \frac{OPT' - 1}{2} \implies OPT > \frac{OPT' - 1}{2}$$

$$2 \cdot OPT > OPT' - 1 \implies 2 \cdot OPT + 1 > OPT' \implies 2 \cdot OPT \geq OPT'$$

Bin Packing – Ratio Proof (Next Fit)

Claim: Next Fit has approximation ratio **<= 2.0**

Key observation: For any two consecutive bins, their combined item sizes exceed C.

- Otherwise, the items in the second bin would have fit in the first.

Proof:

Consider consecutive bin pairs: (bin 1, bin 2), (bin 3, bin 4), …

Each pair has total item size > C. There are at least OPT′/2 such pairs.

$$\sum s_i > \frac{OPT'}{2} \times C$$

$$\frac{\sum s_i}{C} > \frac{OPT'}{2} \implies OPT > \frac{OPT'}{2} \implies 2 \cdot OPT > OPT'$$

4. Job Scheduling Problem

Job Scheduling – Problem Definition

Job Scheduling Problem:

- n jobs with processing times $t_1, t_2, \ldots, t_n$
- m identical machines $M_1, M_2, \ldots, M_m$
- Each job runs on exactly one machine without interruption
- Each machine processes one job at a time

Goal: Assign jobs to machines to **minimize the makespan** (the time when the last job finishes).

Approx_JobScheduling – Algorithm

Input: n jobs with times t_i, m machines

Output: Makespan (latest finishing time)

```
for j = 1 to m:
    L[j] = 0        // finishing time of machine j

for i = 1 to n:
    min = 1
    for j = 2 to m:                  // find earliest-finishing machine
        if L[j] < L[min]:
            min = j
    assign job i to machine M_min
    L[min] = L[min] + t_i

return max(L[1], ... , L[m])
```

Strategy: Always assign the next job to the machine that finishes earliest.

Worked Example – Job Scheduling

Jobs: t = [5, 2, 4, 3, 4, 7, 9, 2, 4, 1], **4 machines**

Step-by-step assignment:

Job	t_i	Assigned to	Machine loads after
1	5	M1	[5, 0, 0, 0]
2	2	M2	[5, 2, 0, 0]
3	4	M3	[5, 2, 4, 0]
4	3	M4	[5, 2, 4, 3]
5	4	M2	[5, 6, 4, 3]
6	7	M4	[5, 6, 4, 10]

Job Scheduling – Approximation Ratio Proof

Claim: Approx_JobScheduling has approximation ratio **<= 2.0**

Let the last job assigned be job i, starting at time T. Then OPT′ = T + t_i.

Two lower bounds on OPT:

1. **OPT >= (sum of all t_i) / m**: Even with perfect load balancing, no machine finishes before the average.

2. **OPT >= t_i**: The optimal solution must also process job i, taking at least t_i time.

Proof:

$$T \leq T' = \frac{\sum_{j \neq i} t_j}{m} \leq \frac{\sum t_j}{m} \leq OPT$$

T is when the earliest machine was free; T′ is the average excluding job i. Since job i went to the earliest machine, T <= T′.

$$OPT' = T + t_i \leq OPT + OPT = 2 \cdot OPT$$

Job Scheduling – Time Complexity

- For each of n jobs, find the minimum among m machines: O(m)
- Finding the maximum at the end: O(m)

Time Complexity: n * O(m) + O(m) = **O(nm)**

5. Clustering Problem

Clustering – Problem Definition

k-Clustering Problem:

- Given n points in a 2D plane and an integer k
- Partition the points into **k groups**, each with a designated **center** point
- **Goal**: Minimize the **maximum group diameter** (the diameter of the largest cluster)

Applications: Recommendation systems, data mining, VLSI design, parallel processing, web search, pattern recognition, gene analysis, social network analysis, and many more.

Farthest-First Traversal – Greedy Strategy

Idea: Select centers one at a time, each time choosing the point **farthest from all existing centers**.

1. Pick an arbitrary point as the first center C_1
2. For the j-th center (j = 2, …, k):
   - For each non-center point x_i, compute D[i] = distance to the **nearest** existing center
   - Select the point with the **largest** D[i] as the next center C_j
3. Assign each non-center point to its nearest center

Intuition: Spreading centers far apart ensures each cluster covers a small area.

Approx_k_Clusters – Algorithm

Input: n points x_0, …, x_{n-1}, number of groups k > 1

Output: k groups with centers

```
C[1] = x_r                      // random first center
for j = 2 to k:
    for i = 0 to n-1:
        if x_i is not a center:
            D[i] = min distance from x_i to any existing center
    C[j] = x_i where D[i] is maximum (and x_i is not a center)

Assign each non-center point to its nearest center
return C and cluster assignments
```

Worked Example – Clustering (k = 4)

Step 1: Pick arbitrary point as C_1

Step 2: Compute D[i] for all points; farthest from C_1 becomes C_2

Step 3: For each point, D[i] = min(dist(x_i, C_1), dist(x_i, C_2))

- Example: D[1]=18, D[2]=19, D[3]=20, D[4]=17 (others < 20)
- Point with D[3]=20 (largest) becomes C_3

Step 4: Recompute D[i] = min(dist to C_1, C_2, C_3); farthest becomes C_4

Step 5: Assign each remaining point to its nearest center

Clustering – Time Complexity

Component	Analysis
Inner for-loop (line 3–5)	Each point computes distance to all centers: O(kn)
Find max D[i] (line 6)	O(n)
Outer for-loop (line 2)	Repeats (k-1) times
Final assignment (line 7)	O(kn)

Total: (k-1) * (O(kn) + O(n)) + O(kn) = **O(k^2 n)**

Clustering – Approximation Ratio Proof

Claim: Approx_k_Clusters has approximation ratio **<= 2.0**

Setup: After finding k centers, imagine selecting a **(k+1)-th virtual center** C_{k+1} using the same farthest-first rule. Let **d** = distance from C_{k+1} to its nearest center.

Step 1: OPT >= d

- We now have (k+1) center points that must be divided into k groups
- By pigeonhole principle, at least two centers share a group
- The diameter of that group >= d, so OPT >= d

Step 2: OPT' <= 2d

- C_{k+1} is the farthest point from any center, so every other point is within distance d of its nearest center
- Each cluster has radius <= d, so diameter <= 2d
- Therefore OPT' <= 2d

Conclusion: 2 * OPT >= 2d >= OPT', so approximation ratio <= **2.0**

Clustering – Practical Considerations

Random first center: Since C_1 is chosen randomly, results vary between runs.

- Run the algorithm multiple times and take the **best result**

Outlier sensitivity: Noisy data or outliers can distort center selection.

- **Preprocess** data to remove outliers before applying the algorithm

Note: This algorithm is closely related to the **k-center problem** and has inspired practical algorithms like **k-means++** initialization.

Approximation Algorithms – Summary Table

Problem	Indirect Optimal	Ratio	Time Complexity
TSP	MST weight M (OPT > M)	<= 2.0	O(n^2) or O(m log m)
Vertex Cover	Matching size |M| (OPT >= |M|)	<= 2.0	O(nm)
Bin Packing (FF/BF/WF)	sum(s_i)/C (OPT >= sum/C)	<= 2.0	O(n^2)
Bin Packing (NF)	sum(s_i)/C (OPT >= sum/C)	<= 2.0	O(n)
Job Scheduling	sum(t_i)/m and max(t_i)	<= 2.0	O(nm)
Clustering	Virtual (k+1)-th center dist d	<= 2.0	O(k^2 n)

All five problems have approximation ratio **2.0** – giving solutions at most twice the optimal.

Part 2. Final Exam Review

Weeks 09–13 Key Concepts

Week 09 – Search Trees

Binary Search Tree (BST)

- Property: left subtree < root < right subtree
- Average case: O(log n) for search, insert, delete
- Worst case: sorted input creates a linked list shape => O(n)

Red-Black Tree (RBT)

- Self-balancing BST with 5 properties (root black, leaves black, red node has black children, etc.)
- Height guarantee: h <= 2 log(n+1) => all operations O(log n)
- Insertion fix: **Restructuring** (uncle is black) or **Recoloring** (uncle is red)

B-Tree

- Wide, shallow tree optimized for **disk I/O** (one node = one disk block)
- Hundreds of keys per node; height ~log_t(n)
- Insertion: split when node is full, promote middle key

Week 10 – Hash Tables

Hash Function: Maps keys to array indices; h(k) = k mod m

Collision Resolution:

- **Separate Chaining**: Each bucket holds a linked list
- **Open Addressing**: Probe for next empty slot
  - Linear probing (clustering problem), Quadratic probing, Double hashing

Load Factor: alpha = n/m (number of items / table size)

- High alpha => more collisions => slower operations
- **Rehashing**: When alpha exceeds threshold, resize table and re-insert all elements

Average-case complexity: O(1) for search, insert, delete

Comparison: Hash table O(1) avg vs BST O(log n) vs Sorted array O(log n) search + O(n) insert

Weeks 11–12 — Graph Algorithms

Graph Traversal:

- **BFS** (Breadth-First Search): Queue-based, O(V+E), shortest path in unweighted graphs
- **DFS** (Depth-First Search): Stack/recursion, O(V+E), cycle detection, topological sort

Minimum Spanning Tree (MST):

- **Kruskal's**: Sort edges, add if no cycle (Union-Find); O(E log E)
- **Prim's**: Grow tree from a vertex, add nearest edge; O(V^2) or O(E log V) with heap

Shortest Path:

- **Dijkstra's**: Single-source, non-negative weights; O(V^2) or O(E log V)
- **Bellman-Ford**: Single-source, handles negative weights; O(VE)
- **Floyd-Warshall**: All-pairs shortest paths; O(V^3)

Topological Sort: Linear ordering of DAG vertices; O(V+E)

Strongly Connected Components (SCC): Kosaraju's algorithm using two DFS passes; O(V+E)

Week 13 – NP-Completeness

Complexity Classes:

- **P**: Problems solvable in polynomial time (e.g., sorting, shortest path, MST)
- **NP**: Problems where a solution can be **verified** in polynomial time
- **NP-hard**: At least as hard as every problem in NP
- **NP-complete**: Both in NP and NP-hard

Key Question: Does P = NP? (Most researchers believe P != NP)

Polynomial-time Reduction: Problem A reduces to Problem B (A <=_p B)

- If B is in P, then A is in P
- If A is NP-hard, then B is NP-hard

Classic NP-complete problems: SAT, 3-SAT, Vertex Cover, TSP, Hamiltonian Cycle, Subset Sum, Bin Packing, Graph Coloring, Clique

Practical significance: If a problem is NP-complete, use approximation algorithms, heuristics, or restrict input size.

Final Review – What to Focus On

Week	Must-Know Topics
09	BST operations & complexity, RBT properties & insertion fix (restructuring vs recoloring), B-Tree structure & split
10	Hash function design, collision resolution methods, load factor & rehashing, O(1) average analysis
11	BFS/DFS traversal order & complexity, Kruskal's & Prim's algorithms, Union-Find basics
12	Dijkstra's vs Bellman-Ford vs Floyd-Warshall, topological sort, SCC (Kosaraju's)
13	P vs NP definition, polynomial reduction concept, identifying NP-complete problems
14	Approximation ratio concept, indirect optimal solution, all five approximation algorithms & their ratio proofs

Summary

- **Approximation algorithms** sacrifice optimality for polynomial-time solvability
- The **approximation ratio** measures solution quality using an indirect optimal bound
- All five problems studied today (TSP, Vertex Cover, Bin Packing, Job Scheduling, Clustering) achieve a ratio of **2.0**
- The common proof technique: find a computable lower/upper bound on OPT, then relate it to APX
- **Final exam** covers Weeks 09–14: search trees, hashing, graphs, NP-completeness, and approximation algorithms

Q & A

codingchild@korea.ac.kr